Report Assignment 5: Reinforcement Learning for Inverted Pendulum Control

Author: Simone Magistrali

Email: smi25011@student.mdu.se

Course: Learning Systems DVA493

Professor: Ning Xiong

Semester: Fall 2025

1. Introduction and Objective

The objective of this assignment is to apply Reinforcement Learning (RL), specifically Model-Free Q-Learning, to solve the automatic control problem of an inverted pendulum system. The agent must learn the force ("right" or "left") to apply to the trolley to keep the hinged pole in vertical equilibrium, preventing it from falling or the trolley from exceeding the limits of the rail. The requirement is that the system survive (do not fail) for at least one minute.

2. State Characterization (Discretization)

The physical system is described by continuous variables, but to apply the tabular Q-Learning algorithm it is necessary to discretize the state space.
The state of the world is defined by four continuous variables:

1. x: Cart position (meters).
2. Cart speed (m/s).
3. theta: Pole angle (radians/degrees).
4. Angular velocity of the pole (radians/s).

In the getState.m file, the continuous space has been binning. The boundaries and dimensions of the binning have been defined.

- **Position (** x **):** Divided into **10** intervals (using limits from -2.4 to 2.4 m).
- **Speed:** Divided into **5** ranges (using limits from -2 to 2 m/s).
- **Angle (** theta **):** Divided into **20** intervals. The angle is converted to degrees and discretized between -12° and 12°.
- **Angular velocity:** Divided into **5** intervals (using limits from -2 to 2).

The total number of possible discrete states in the Q-Table is then:
10 * 5 * 20 * 5 = 5000 states
Each combination of these discretized variables corresponds to a unique index in the Q-Table.

3. Reward Function

The design of the reward function is crucial to guiding the agent's learning. In the getRewardAndFailure.m file, the implemented logic is as follows:

- **Failure Conditions:** The system fails if:
  - |x| > 2.4 meters (the trolley leaves the tracks).
  - |theta| > 12 degrees (the pole falls too far).
- **Reward Allocation:**
  - **Penalty:** If the system enters a failure state, a significant negative penalty is assigned: r = -100.
  - **Survival:** For each step in which the system remains within the safety limits (does not fail), a positive reward is assigned: r = +1.

This incentivizes the agent to maximize the duration of the episode, accumulating +1 for each instant of time he manages to balance the pole.

4. Function to Learn

The function that the agent must learn is the Action-Value Function, denoted as Q(s, a).
This function estimates the "value" (discounted sum of future rewards) of taking a specific action while being in a given state s.

- **States (** s **):** One of the 5000 discrete states described in section 2.
- **Actions (** a **):** Two possible discrete actions defined in the system:
  1. Apply force -10 Newton (Left).
  2. Apply force +10 Newton (Right).

The goal is to find the optimal Q^*(s, a) that maximizes the total expected reward. Once the Q table is learned, the optimal control policy will simply be to choose the action with the highest Q value in each state:
a = max_a Q(s, a)

5. Learning Algorithm (Q-Learning)

The algorithm used is **Q-Learning , a** *model-free* and *off-policy* reinforcement learning method . The implementation in main.m follows these steps:

1. **Initialization:** The Q-Table is initialized (or loaded from a previous training).
2. **Training Cycle:** The agent runs 10,000 episodes.
3. **Update:** At each time step (t), the agent observes the current state s_t, chooses an action a_t, observes the reward r_{t+1} and the new state s_{t+1}.
4. **Update Rule** :
   The value Q(s_t, a_t) is updated according to the formula:
   - Q(s_t, a_t) <- r_{t+1} + gamma * max_{a'} Q(s_{t+1}, a')
   - If state s_{t+1} is terminal (failure), the future term is 0 and the target is only the penalty (-100).
   - The discount factor (gamma) is set to **0.99** , indicating that the agent places great importance on long-term survival.

At the end of training, the resulting Q-Table is saved in Q_Table_Addestrata.mat.

6. Exploration Strategy

During learning, it's necessary to balance exploration (trying new actions) and exploitation (using what you've learned). In the main.m file, a probabilistic selection strategy controlled by the kappa parameter was used.

The probability of choosing an action is calculated as:

$$P(a) = \frac{k^{Q(s,a)}}{\sum_a k^{Q(s,a)}}$$

- Q values are normalized by subtracting the maximum (for numerical stability).
- Initially, kappa is low (10) for increased exploration. During episodes, kappa progressively increases (kappa = kappa $\times$ 1.0005) up to 20. As kappa increases, the agent becomes more "greedy", favoring exploitation as knowledge consolidates.

7. Simulation Results

To verify the effectiveness of the learned strategy, the agent was tested using the testAgent.m script, which loads the trained Q-Table and simulates the pendulum control for up to 3000 steps (60 seconds, since each step is 0.02s).

The following figures show the progression of the trolley position and the pole angle during the test.

Cart Position Chart (x)

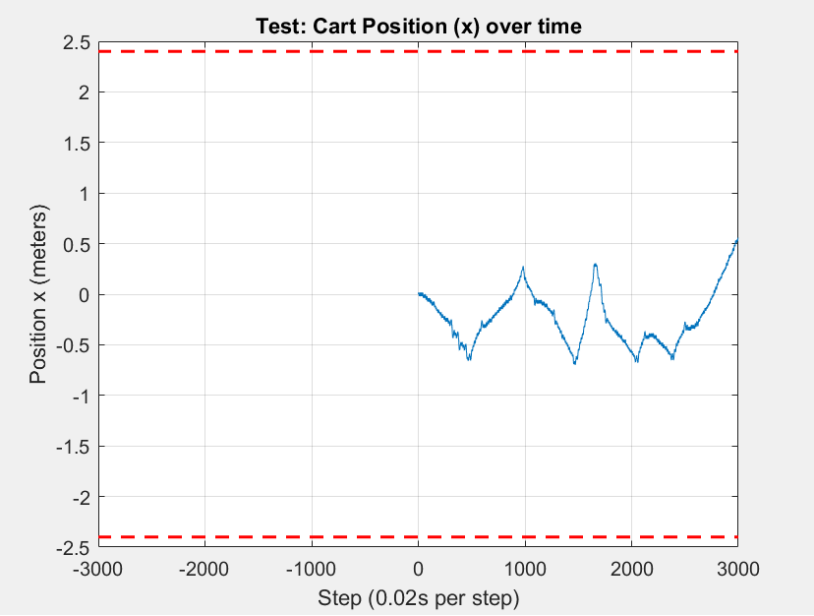


Figure 1: Position of the trolley over time. The dashed red lines indicate the safety limits (+-2.4 m).

Pole Angle Graph (theta)

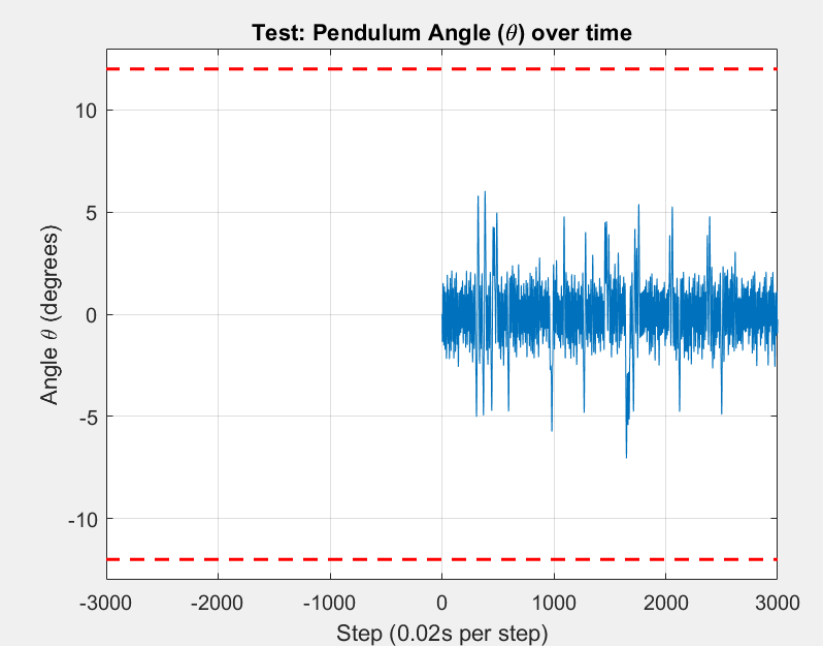


Figure 2: Pile angle over time. The dashed red lines indicate safety limits (+-12 degrees).

Discussion

As shown in the graphs, the agent stabilized the system throughout the 3,000 simulation steps, maintaining the x position and theta angle within the limits. The oscillation of the angle around zero confirms that the agent learned to counteract the falling pole by applying the correct forces.

8. Conclusion

A control system for an inverted pendulum based on reinforcement learning (Q-learning) with a discretized state space (5,000 combinations) has been successfully implemented. The agent learned an effective policy, balancing the pendulum for over a minute and meeting the design requirements.